Vision Mamba for Diabetic Retinopathy Grading: a From-Scratch Implementation and Cross-Dataset Study

Marc Pedrin
IISc Spectrum Lab internship task

November 3, 2026

Abstract

We implement Vision Mamba (Vim) [11], a vision backbone built from bidirectional selective state space models, from scratch in plain PyTorch, and apply it to five-class diabetic retinopathy (DR) grading. Every component, including the selective scan, is written explicitly; no Mamba kernels, pretrained weights or ready-made data loaders are used. The model is trained on EyePACS and evaluated both on a held-out, patient-disjoint EyePACS test split and, without any adaptation, on the external Messidor-2 dataset. We use a scaled-down configuration (128-pixel inputs, 12 blocks) suited to a single GPU, and document every deviation from the paper. After 25 epochs on one GPU the model reaches a quadratic weighted kappa of 0.119 on the EyePACS test split and 0.083 on Messidor-2: it learns a weak but real ordering of the grades, which partly transfers to the new domain. We also show that the model collapses to a single grade (kappa 0) on a contrast-enhanced copy of Messidor-2, and report how a first, failed training run led us to restore the $D \cdot x$ skip term of the official Mamba code.

1 Introduction

Diabetic retinopathy is damage to the retina caused by diabetes, and a leading cause of preventable blindness in working-age adults. It progresses silently, so diabetic patients are screened regularly with fundus photographs, colour images of the back of the eye. Screening at population scale needs many trained graders, which motivates automated grading.

Grading scale. We use the five-level international clinical scale [10], as in the EyePACS and Messidor-2 labels:

Grade	Name	Typical findings
0	No DR	no abnormalities
1	Mild non-proliferative	microaneurysms only
2	Moderate non-proliferative	more than microaneurysms, less than grade 3
3	Severe non-proliferative	extensive haemorrhages, venous beading, IRMA
4	Proliferative	new vessel growth and/or vitreous haemorrhage

The grades are *ordinal*: confusing grade 0 with grade 4 is a much worse error than confusing grade 1 with grade 2. The lesions that separate the low grades are tiny (a microaneurysm spans a few pixels at our resolution), and the classes are heavily imbalanced, with about 73% of EyePACS images at grade 0. Both facts shape our preprocessing, our loss and our choice of metric.

Why Vision Mamba. Vision Transformers [4] relate every image patch to every other through self-attention, at a cost quadratic in the number of patches. Vim replaces attention with selective state space models (Mamba [6]), whose cost is linear in the sequence length, and makes them bidirectional so that every patch can receive information from both sides.

Contributions of this project. (i) A line-by-line, from-scratch implementation of the Vim block (Algorithm 1 of the paper), tested against hand-computed cases; (ii) a leakage-free, patient-level EyePACS pipeline with fundus-specific preprocessing; (iii) from-scratch metrics, including quadratic weighted kappa, verified against scikit-learn; (iv) an in-domain and cross-domain evaluation.

2 The Vision Mamba architecture

2.1 State space models

A state space model maps an input signal $x(t)$ to an output $y(t)$ through a hidden state $h(t) \in \mathbb{R}^N$:

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \qquad y(t) = \mathbf{C}h(t).$$

This continuous form is theory only and is never coded. To process a sequence of tokens it is discretised with a step size Δ, using the zero-order hold:

$$\overline{\mathbf{A}} = \exp(\Delta\mathbf{A}), \qquad \overline{\mathbf{B}} = (\Delta\mathbf{A})^{-1}\big(\exp(\Delta\mathbf{A}) - \mathbf{I}\big) \cdot \Delta\mathbf{B}, \tag{1}$$

which gives the recurrence

$$h_t = \overline{\mathbf{A}}\, h_{t-1} + \overline{\mathbf{B}}\, x_t, \qquad y_t = \mathbf{C}\, h_t. \tag{2}$$

When $\overline{\mathbf{A}}$, $\overline{\mathbf{B}}$ and $\mathbf{C}$ are the same at every step (as in S4 [7]), the recurrence can be unrolled into a single global convolution with kernel

$$\overline{\mathbf{K}} = \big(\mathbf{C}\overline{\mathbf{B}},\ \mathbf{C}\overline{\mathbf{A}}\,\overline{\mathbf{B}},\ \ldots,\ \mathbf{C}\overline{\mathbf{A}}^{M-1}\overline{\mathbf{B}}\big), \qquad y = x * \overline{\mathbf{K}}. \tag{3}$$

Equation numbers (1)–(3) coincide with the paper's Eqs. 1–3.

Selectivity. Mamba makes $\mathbf{B}$, $\mathbf{C}$ and Δ *functions of the current token*, computed by small linear layers. Each token can then decide how much to write into the state, what to read out of it, and how much of the past to forget (a large Δ drives $\overline{\mathbf{A}}$ towards zero). Because these quantities change at every step, no fixed kernel $\overline{\mathbf{K}}$ exists and Eq. (3) no longer applies: the recurrence (2) must be run, which we do with an explicit loop. $\mathbf{A}$ is diagonal and kept strictly negative, so $\overline{\mathbf{A}} = \exp(\Delta\mathbf{A}) \in (0, 1)$ and the state decays rather than explodes.

2.2 From images to token sequences

An image of size $H \times W$ is cut into J patches of $P \times P$ pixels, each linearly projected to D dimensions. A learnable class token t_{cls} is added and learnable position embeddings $\mathbf{E}_{\text{pos}}$ are summed in:

$$\mathbf{T}_0 = [\, t_{\text{cls}};\ t_p^1\mathbf{W};\ t_p^2\mathbf{W};\ \ldots;\ t_p^J\mathbf{W}\,] + \mathbf{E}_{\text{pos}}. \tag{4}$$

(t_{cls} is printed at the front, but the paper's default model places it in the middle of the sequence; see Section 2.4.) The sequence passes through L Vim blocks with residual connections, $\mathbf{T}_l = \text{Vim}(\mathbf{T}_{l-1}) + \mathbf{T}_{l-1}$, and the class token's final output is normalised and classified: $f = \text{Norm}(\mathbf{T}_L^0)$, $\hat{p} = \text{MLP}(f)$.

2.3 The bidirectional Vim block

Algorithm 1 reproduces the paper's Algorithm 1, with the same line numbering (comment lines included). Our `vim/vim_block.py` and `vim/selective_scan.py` follow it line by line, and each line of code is annotated with the algorithm line it implements.

Algorithm 1 Vim block process (after Algorithm 1 of Zhu et al. [11]).

Require: token sequence $\mathbf{T}_{l-1} : (\mathtt{B}, \mathtt{M}, \mathtt{D})$
Ensure: token sequence $\mathbf{T}_{l} : (\mathtt{B}, \mathtt{M}, \mathtt{D})$
1: */* normalise the input sequence* $\mathbf{T}'_{l-1}$ **/*
2: $\mathbf{T}'_{l-1} : (\mathtt{B}, \mathtt{M}, \mathtt{D}) \leftarrow \mathbf{Norm}(\mathbf{T}_{l-1})$
3: $\mathbf{x} : (\mathtt{B}, \mathtt{M}, \mathtt{E}) \leftarrow \mathbf{Linear}^{\mathbf{x}}(\mathbf{T}'_{l-1})$
4: $\mathbf{z} : (\mathtt{B}, \mathtt{M}, \mathtt{E}) \leftarrow \mathbf{Linear}^{\mathbf{z}}(\mathbf{T}'_{l-1})$
5: */* process with different direction */*
6: **for** o in {forward, backward} **do**
7: $\quad \mathbf{x}'_o : (\mathtt{B}, \mathtt{M}, \mathtt{E}) \leftarrow \mathbf{SiLU}(\mathbf{Conv1d}_o(\mathbf{x}))$
8: $\quad \mathbf{B}_o : (\mathtt{B}, \mathtt{M}, \mathtt{N}) \leftarrow \mathbf{Linear}^{\mathbf{B}}_o(\mathbf{x}'_o)$
9: $\quad \mathbf{C}_o : (\mathtt{B}, \mathtt{M}, \mathtt{N}) \leftarrow \mathbf{Linear}^{\mathbf{C}}_o(\mathbf{x}'_o)$
10: $\quad$ */* softplus ensures positive* Δ_o **/*
11: $\quad \Delta_o : (\mathtt{B}, \mathtt{M}, \mathtt{E}) \leftarrow \log\big(1 + \exp(\mathbf{Linear}^{\Delta}_o(\mathbf{x}'_o) + \mathbf{Parameter}^{\Delta}_o)\big)$
12: $\quad$ */* shape of* $\mathbf{Parameter}^{\mathbf{A}}_o$ *is* $(\mathtt{E}, \mathtt{N})$ **/*
13: $\quad \overline{\mathbf{A}}_o : (\mathtt{B}, \mathtt{M}, \mathtt{E}, \mathtt{N}) \leftarrow \Delta_o \otimes \mathbf{Parameter}^{\mathbf{A}}_o$
14: $\quad \overline{\mathbf{B}}_o : (\mathtt{B}, \mathtt{M}, \mathtt{E}, \mathtt{N}) \leftarrow \Delta_o \otimes \mathbf{B}_o$
15: $\quad$ */* initialise* $\mathbf{h}_o$ *and* $\mathbf{y}_o$ *with 0 */*
16: $\quad \mathbf{h}_o : (\mathtt{B}, \mathtt{E}, \mathtt{N}) \leftarrow \mathrm{zeros}(\mathtt{B}, \mathtt{E}, \mathtt{N})$
17: $\quad \mathbf{y}_o : (\mathtt{B}, \mathtt{M}, \mathtt{E}) \leftarrow \mathrm{zeros}(\mathtt{B}, \mathtt{M}, \mathtt{E})$
18: $\quad$ */* SSM recurrent */*
19: $\quad$ **for** i in $\{0, \ldots, M-1\}$ **do**
20: $\quad\quad \mathbf{h}_o = \overline{\mathbf{A}}_o[:, i, :, :] \odot \mathbf{h}_o + \overline{\mathbf{B}}_o[:, i, :, :] \odot \mathbf{x}'_o[:, i, :, \mathrm{None}]$
21: $\quad\quad \mathbf{y}_o[:, i, :] = \mathbf{h}_o \otimes \mathbf{C}_o[:, i, :]$
22: $\quad$ **end for**
23: **end for**
24: */* get gated* $\mathbf{y}_o$ **/*
25: $\mathbf{y}'_{\text{forward}} : (\mathtt{B}, \mathtt{M}, \mathtt{E}) \leftarrow \mathbf{y}_{\text{forward}} \odot \mathbf{SiLU}(\mathbf{z})$
26: $\mathbf{y}'_{\text{backward}} : (\mathtt{B}, \mathtt{M}, \mathtt{E}) \leftarrow \mathbf{y}_{\text{backward}} \odot \mathbf{SiLU}(\mathbf{z})$
27: */* residual connection */*
28: $\mathbf{T}_l : (\mathtt{B}, \mathtt{M}, \mathtt{D}) \leftarrow \mathbf{Linear}^{\mathbf{T}}(\mathbf{y}'_{\text{forward}} + \mathbf{y}'_{\text{backward}}) + \mathbf{T}_{l-1}$

Line 13 is printed without the exponential of Eq. (1); we apply it, as the official code does, since otherwise $\overline{\mathbf{A}}$ would be negative. Line 14 is the simplified discretisation $\overline{\mathbf{B}} = \Delta\mathbf{B}$, a first-order approximation of Eq. (1) that is also used by the official Mamba code. The backward direction is implemented by flipping the sequence, running the same code with the backward weights, and flipping the result back.

2.4 Design choices validated by the paper's ablations

Bidirectionality (paper Table 4). A single forward scan only carries information left to right, so a token never hears from the patches after it.

Bidirectional strategy	ImageNet top-1 (%)	ADE20K mIoU
None (forward only)	73.2	32.3
Bidirectional layer	70.9	33.6
Bidirectional SSM	72.8	33.2
Bidirectional SSM + Conv1d (default)	**73.9**	**35.9**

Class token position (paper Table 5). Because the scans are directional and their memory decays with distance, a class token in the middle of the sequence is reached by both directions with the shortest average distance to every patch.

Classification design	ImageNet top-1 (%)
Mean pool	73.9
Max pool	73.4
Head class token	75.2
Double class token	74.3
Middle class token (default)	**76.1**

We use the default variant in both cases: bidirectional SSM with Conv1d, and a middle class token at index 32 of the 65-token sequence.

3 Implementation details and deviations from the paper

Everything is written from scratch in PyTorch: patch embedding, tokens and position embeddings, the selective scan (an explicit Python loop over the sequence), the bidirectional block, the model, the datasets, the split and the metrics. PyTorch supplies only generic building blocks (`nn.Linear`, `nn.Conv1d`, `nn.LayerNorm`, autograd, AdamW) and torchvision only image transforms. The paper's hardware-aware parallel scan (its Section 3.5) is deliberately out of scope.

Setting	Paper (Vim-Ti)	Ours
Input size	224×224	128×128
Patch size / patches	16 / 196	16 / 64
Sequence length M	197	65
Embedding dim D	192	192
Expanded dim E	384	384
State dim N	16	16
Depth L	24	12
Conv1d kernel	4	4
Class token	middle	middle (index 32)
Δ projection	low-rank (code)	full $E \times E$ (as printed)
Norm	RMSNorm (code)	LayerNorm
$D \cdot x$ skip term	yes (code)	yes (not in Algorithm 1, see below)
Parameters	≈7M	6,866,309

Parameter count. Halving the depth does not halve the parameter count, because the printed Algorithm 1 projects Δ with a full $E \times E$ matrix (147,456 weights per direction per block), where the official code uses a rank-12 factorisation. Our block has 557,952 parameters, roughly twice an official block, and the two Δ projections account for 53% of it.

Initialisation. $\mathbf{A} = -\exp(\mathbf{A}_{\log})$ with $\mathbf{A}_{\log}$ initialised to $\log(1, \ldots, 16)$ (S4D-real [8]), giving a spread of decay rates; the Δ bias is set so that the initial step sizes are log-uniform in $[0.001, 0.1]$, as in Mamba; the class token and position embeddings use a truncated normal with standard deviation 0.02.

The $D \cdot x$ skip term, and why we added it. The printed Algorithm 1 has no skip term inside the SSM, while the official Mamba and Vim code compute $y_t = \mathbf{C}h_t + D \odot x'_t$ with a learnable $D \in \mathbb{R}^E$ initialised to ones. Our first full training run followed the printed algorithm exactly and failed: validation QWK stayed at 0 for two epochs and the loss became NaN in epoch 3. We diagnosed the first problem on CPU. With the small initial step sizes ($\Delta \in [0.001, 0.1]$), the state h barely accumulates anything, so without the skip term each block adds almost nothing to its input (output standard deviation about 4×10^{-4} for a unit-variance input). The class

token, which starts identical for every image, therefore stays almost identical: across a batch of different images its final representation varied by only 0.0011 (standard deviation), against 0.56 for the patch tokens. The classifier saw nearly the same input for every image. With the $D \cdot x$ term each token's own value passes straight through the SSM branch, and the spread of the class token rose to 0.18 at initialisation. On a 10-image overfitting test the loss after 25 steps fell to 0.69 with the skip term against 2.65 without it. The term adds 9,216 parameters (2×384 per block). It can be switched off with `model.d_skip=false`.

Efficiency of the reference loop. Indexing the precomputed $\overline{\mathbf{A}}$ inside the loop makes the backward pass allocate a full-size gradient at every step. Splitting the tensors once with `torch.unbind` leaves the mathematics unchanged and made the backward pass about 11× faster on CPU.

Verification. 63 unit and integration tests cover, among others: a two-step selective scan computed by hand; the limits of instant forgetting and of no forgetting (a running sum); separate parameters per direction and a control experiment showing the block is direction-aware; gradient flow through 12 blocks; patient-disjoint splits; and every metric against scikit-learn.

4 Datasets and preprocessing

EyePACS (Kaggle Diabetic Retinopathy Detection, 2015 [5]): 35,126 labelled images from 17,563 patients (left and right eye each), from many camera models. We split it at the *patient* level into 80% train, 10% validation and 10% test, stratified by each patient's worse-eye grade, with a fixed seed. Keeping both eyes of a patient in the same split prevents data leakage: the two eyes are highly correlated, and a model could otherwise score well partly by recognising the patient.

Messidor-2 [3, 1]: 1,748 images from 874 examinations, acquired in French hospitals with a different camera, with the adjudicated grades of Krause et al. [9]. Ungradable images are excluded. Messidor-2 is used *only* for evaluation; it never influences training, checkpoint selection or any hyperparameter.

Preprocessing. (1) Crop the black border to the bounding box of the retina (pixels brighter than 10/255); (2) pad to a square with black so the round retina is not distorted; (3) resize to 128×128; (4) normalise with the ImageNet channel statistics. Training images are additionally flipped horizontally and vertically, rotated by up to $\pm 15°$, and jittered by up to 10% in brightness, contrast and saturation (no hue change, since lesion colour is diagnostic). We deliberately avoid the paper's random crops, random erasing and mixup: they can remove or blend away the few tiny lesions that define the lower grades, silently making the label wrong.

5 Training methodology

Hyperparameter	Paper (ImageNet)	Ours (EyePACS)
Optimiser	AdamW ($\beta_1 = 0.9$)	AdamW ($\beta = (0.9, 0.999)$)
Learning rate	1×10^{-3}	5×10^{-4} (see text)
Weight decay	0.05	0.05 (not on norms, biases, $\mathbf{E}_{\text{pos}}$, t_{cls}, $\mathbf{A}_{\log}$)
Schedule	cosine	3 warmup epochs, cosine to 10^{-5}
Epochs	300	25
Batch size	1024	32
Loss	CE + label smoothing	CE with inverse-frequency class weights
Augmentation	crop, flip, mixup, erasing	flips, rotation $\pm 15°$, mild jitter
EMA	yes	no
Gradient clipping	–	max norm 1.0
Precision	–	float32 (see text)
Checkpoint selection	–	best validation QWK
Seed	–	42 (all generators, deterministic cuDNN)
Hardware	8 A800 GPUs	1 Kaggle T4

The short schedule and small batch reflect a limited GPU budget; the goal of the project is a faithful, understandable implementation rather than a state-of-the-art score. Class weights $w_c = N/(K\, n_c)$ counter the imbalance, so that each grade contributes about equally to the loss.

Stabilising training. The failed first run (Section 3) used the paper's learning rate of 10^{-3} with float16 mixed precision, and its loss became NaN at the end of the 3-epoch warmup, exactly when the learning rate reached its peak. For the final run we halved the learning rate, trained in float32 (the explicit scan multiplies up to 65 factors in a row, which is fragile in float16) and added a guard that skips any batch with a non-finite loss and stops the run if more than half the batches of an epoch are skipped. No batch was skipped in the final run. An epoch took about 9 minutes on a T4 (25 epochs in about 3.8 hours).

Metrics. We report accuracy, macro-averaged precision, recall and F1, and quadratic weighted kappa (QWK) [2], the official EyePACS metric:

$$\kappa = 1 - \frac{\sum_{i,j} W_{ij} O_{ij}}{\sum_{i,j} W_{ij} E_{ij}}, \qquad W_{ij} = \frac{(i-j)^2}{(K-1)^2},$$

where O is the confusion matrix and E the matrix expected by chance from the marginal grade frequencies. The quadratic weights punish a prediction four grades away sixteen times as hard as a prediction one grade away, matching the ordinal nature of DR grades. Accuracy alone is misleading here: always predicting grade 0 already scores about 73%, but has $\kappa = 0$.

6 Results

The checkpoint with the best validation QWK (epoch 17, QWK 0.143) is evaluated once on the EyePACS test split and on Messidor-2. All numbers are also stored in `results/results.json`.

Figure 1 shows slow but steady learning. The training loss falls from 1.64 to 1.55 and the validation loss from 1.59 to 1.54; for reference, a model that outputs equal probabilities scores $\ln 5 \approx 1.61$ under our class-weighted loss. Both losses are still falling at epoch 25, so the model is

Table 1: Results on the held-out EyePACS test split (in-domain) and on Messidor-2 (external, never seen in training).

Metric	EyePACS test	Messidor-2
Images	3,512	1,744
Accuracy	0.465	0.317
QWK	0.119	0.083
Macro precision	0.222	0.206
Macro recall	0.237	0.210
Macro F1	0.212	0.184

Table 2: Per-grade recall (share of each true grade that was predicted correctly).

Grade	EyePACS test	Messidor-2
0 none	0.534	0.344
1 mild	0.061	0.070
2 moderate	0.419	0.501
3 severe	0.129	0.133
4 proliferative	0.040	0.000

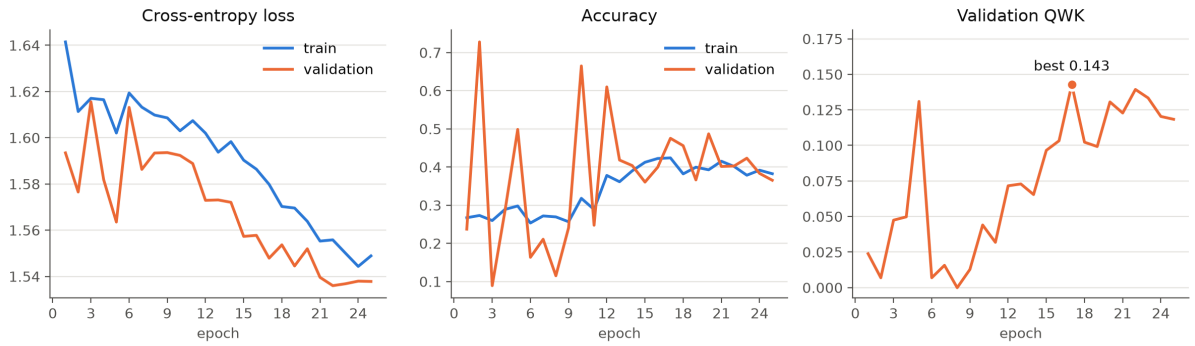


Figure 1: Convergence: training and validation loss and accuracy, and validation QWK per epoch; the marked point is the checkpoint used for evaluation.

under-trained rather than over-fitted: there is no train/validation gap (the validation loss is even lower, since validation images are not augmented). In the first epochs the validation accuracy swings between 0.09 and 0.73 as the predictions jump between favouring one grade and another; it settles once the learning rate decays. Validation QWK rises from about 0.02 to a peak of 0.143 at epoch 17 and stays near 0.12–0.14 afterwards.

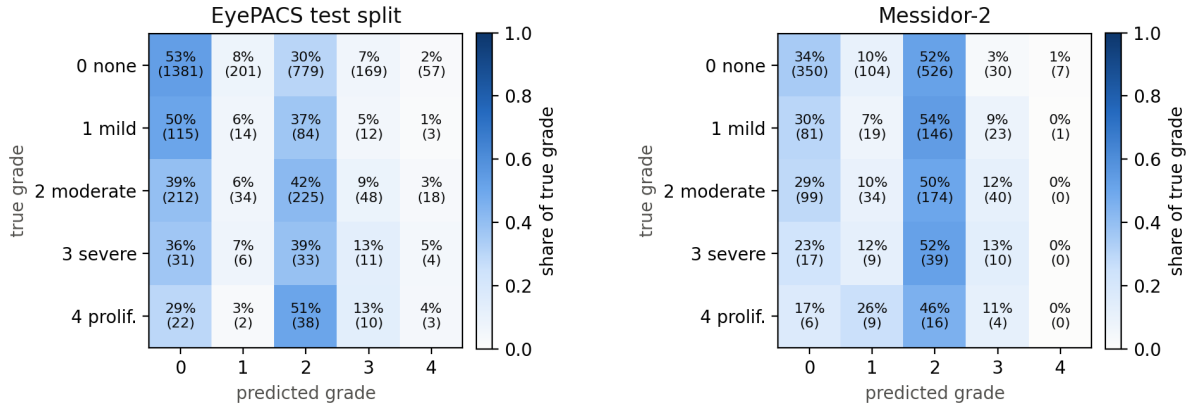


Figure 2: Row-normalised confusion matrices on the EyePACS test split (left) and Messidor-2 (right). Cell colour is the share of the true grade; the count is in brackets.

On EyePACS (Fig. 2, left) the most common error is between grades 0 and 2: 30% of healthy eyes are called moderate, and 39% of moderate eyes are called healthy. Grade 1 is almost never recognised (6% recall); its only lesions, microaneurysms, are one or two pixels wide at 128×128. Most severe and proliferative eyes are predicted as grade 2, an error of one or two grades that QWK penalises only mildly, which is why the kappa is positive despite the low accuracy. The class weights (0.27 for grade 0, 1.32 for grade 2) push the model away from the majority grade, trading accuracy for recall on the diseased grades.

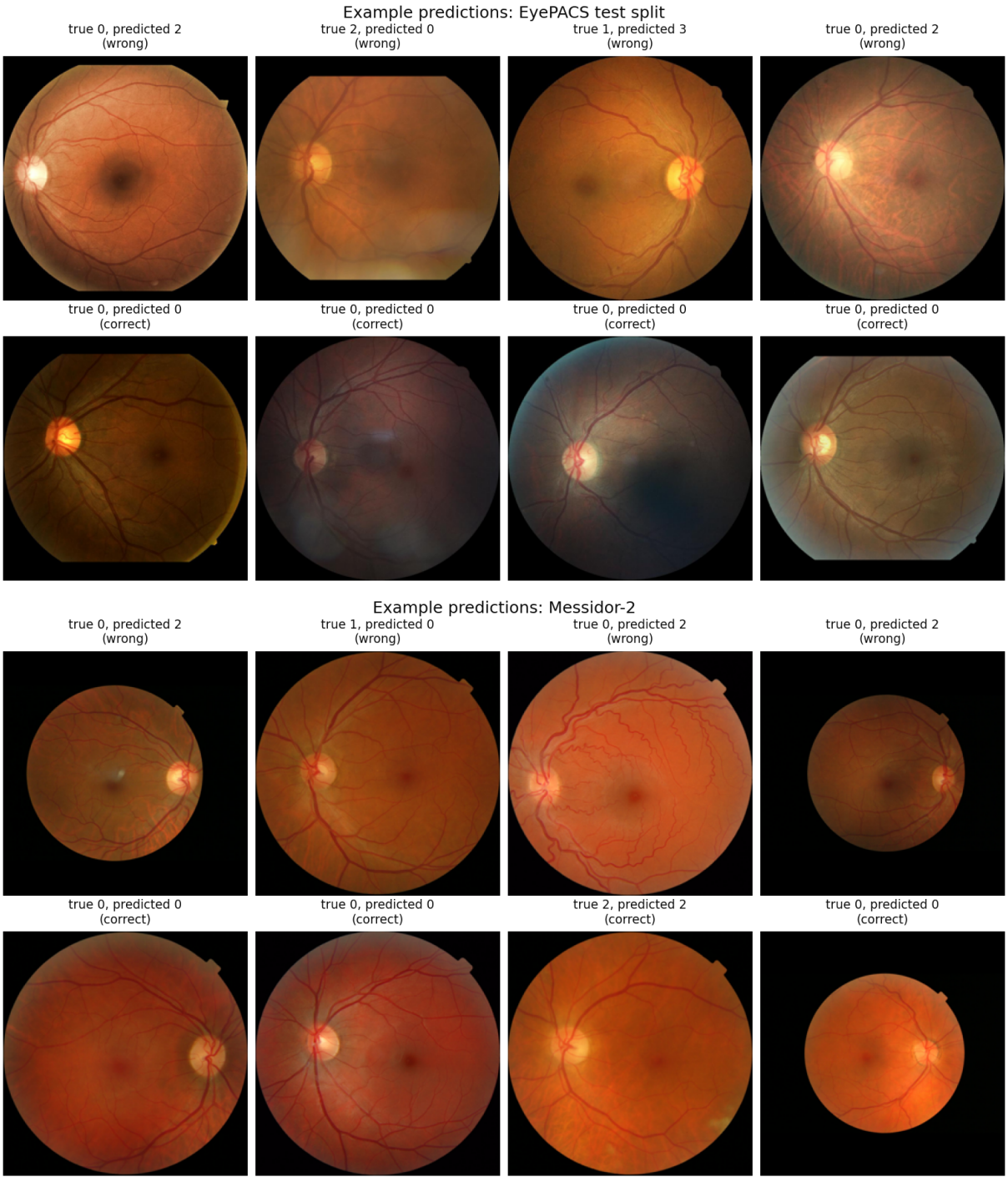


Figure 3: Example predictions on EyePACS (top) and Messidor-2 (bottom): true and predicted grade, half chosen among errors.

7 Analysis of cross-dataset generalisation

A drop in performance from the EyePACS test split to Messidor-2 is expected, even though the task is the same, because of *domain shift*: Messidor-2 was acquired with a different camera (colour balance, sharpness, field of view and border shape differ), in a different population (French hospital patients rather than a mostly US screening network), under different acquisition protocols, and graded to a different, adjudicated reference standard. Whatever the model learnt that is specific to EyePACS photographs rather than to the lesions themselves does not transfer. Our preprocessing (cropping to the retina, padding, normalisation, colour jitter) is designed to reduce the most obvious of these differences, but cannot remove them.

Size of the drop. QWK falls from 0.119 to 0.083 and macro F1 from 0.212 to 0.184 (Table 1). The model keeps some of its ordering ability on the new camera and population, but loses about a third of it. Accuracy drops more (0.465 to 0.317), partly because Messidor-2 has fewer healthy eyes (58% grade 0 against 74% in the EyePACS test split) and more mild ones (15% against 6%).

Which grades suffer. The model over-calls disease on Messidor-2: it predicts grade 2 for 52% of all Messidor-2 images, against 33% on EyePACS, and for 52% of the healthy eyes. Grade-0 recall therefore falls from 0.53 to 0.34, while grade-2 recall rises from 0.42 to 0.50. Grades 1 and 3 stay at about the same low recall, and none of the 35 proliferative eyes is recognised. A shift towards one grade like this is typical of domain shift: the brighter, sharper and more uniformly lit Messidor-2 photographs look different from most EyePACS images, and the model reads part of that difference as signs of disease.

Sensitivity to preprocessing. Our first Messidor-2 evaluation used a publicly shared, already preprocessed copy of the images (grey background, strong local contrast enhancement). On that copy the same model predicts grade 0 for every one of the 1,744 images: accuracy 0.583 (the share of grade 0) and QWK exactly 0. On the original photographs, the numbers above, it uses all grades. The labels, the model and our own preprocessing are identical in the two runs; only the input photographs differ. This shows how strongly a model trained from scratch on one image style depends on that style, and that the preprocessing applied to an external test set must match the one used in training before cross-dataset numbers can be compared.

8 Limitations

- **Scale.** 128-pixel inputs discard detail: microaneurysms become a pixel or two, which particularly hurts grade 1. Twelve blocks and 25 epochs are far below the paper's 24 blocks and 300 epochs, and the losses were still falling when training stopped.
- **Speed.** The selective scan is a sequential Python loop, much slower than the paper's fused parallel kernel; this limited the training budget.
- **No pretraining.** The model is trained from random initialisation on about 28,000 images, whereas Vim-Ti is trained on 1.28 million ImageNet images.
- **Label noise.** EyePACS grades come from single graders and are known to be noisy, which caps the achievable agreement.
- **Single run.** Results are from one seed; we did not measure the variance across seeds. With validation QWK moving by up to 0.04 between consecutive epochs, differences of that size should not be over-read.

- **Faithfulness choices.** Following the printed Algorithm 1 (full Δ projection, LayerNorm) makes our blocks larger than the official ones and our model a slightly different function from the official Vim-Ti. The $D \cdot x$ term is the one place where we followed the official code instead, because training failed without it.

9 Conclusion

We implemented Vision Mamba from first principles, keeping a direct, documented correspondence between the paper's equations and Algorithm 1 and the code, and applied it to diabetic retinopathy grading with a leakage-free evaluation on an in-domain test split and an external dataset. Trained from scratch for 25 epochs at 128×128, the model reaches a QWK of 0.119 on the EyePACS test split and 0.083 on Messidor-2: weak, but clearly above chance, and with a domain-shift drop of about a third. The results are limited mainly by the training budget, the resolution and the lack of pretraining, and the still-falling losses suggest that longer training alone would help. The main lessons are that the selective, bidirectional scan can be implemented transparently in a few dozen lines; that details left out of the printed algorithm, such as the $D \cdot x$ skip term, can decide whether the model trains at all; and that cross-dataset evaluation, including how the external images were preprocessed, is essential for judging a medical imaging model.

References

[1] Michael D. Abràmoff, James C. Folk, Dennis P. Han, Jonathan D. Walker, David F. Williams, Stephen R. Russell, Pascale Massin, Béatrice Cochener, Philippe Gain, Li Tang, Mathieu Lamard, Daniela C. Moga, Gwénolé Quellec, and Meindert Niemeijer. Automated analysis of retinal images for detection of referable diabetic retinopathy. *JAMA Ophthalmology*, 131(3):351–357, 2013.

[2] Jacob Cohen. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4):213–220, 1968.

[3] Etienne Decencière, Xiwei Zhang, Guy Cazuguel, Bruno Lay, Béatrice Cochener, Caroline Trone, Philippe Gain, Richard Ordonez, Pascale Massin, Ali Erginay, Béatrice Charton, and Jean-Claude Klein. Feedback on a publicly distributed image database: the Messidor database. *Image Analysis & Stereology*, 33(3):231–234, 2014.

[4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[5] EyePACS and California Healthcare Foundation. Diabetic retinopathy detection. Kaggle competition, `https://www.kaggle.com/c/diabetic-retinopathy-detection`, 2015.

[6] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[7] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022.

[8] Albert Gu, Ankit Gupta, Karan Goel, and Christopher Ré. On the parameterization and initialization of diagonal state space models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[9] Jonathan Krause, Varun Gulshan, Ehsan Rahimy, Peter Karth, Kasumi Widner, Greg S. Corrado, Lily Peng, and Dale R. Webster. Grader variability and the importance of reference standards for evaluating machine learning models for diabetic retinopathy. *Ophthalmology*, 125(8):1264–1272, 2018.

[10] C. P. Wilkinson, Frederick L. Ferris, Ronald E. Klein, Paul P. Lee, Carl David Agardh, Matthew Davis, Diana Dills, Anselm Kampik, R. Pararajasegaram, and Juan T. Verdaguer. Proposed international clinical diabetic retinopathy and diabetic macular edema disease severity scales. *Ophthalmology*, 110(9):1677–1682, 2003.

[11] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.09417.